

Rug Royale: Architecture Diagram & Requirements

Turbin3 Capstone · Group Assignment

2026-09-29

- [0. Team and roles](#)
- [1. Actors \(carried forward from the LOI\)](#)
- [2. MVP use cases](#)
- [3. Granularity self-check \(team, before any AI\)](#)
- [4. Adversarial analysis \(AI second pass\)](#)
- [5. Requirements \(REQ IDs\)](#)
- [6. Architecture diagrams](#)
- [7. Design decisions and known MVP limits](#)
- [8. On-chain requirements matrix](#)
- [9. Traceability](#)

o. Team and roles

Member	MVP role	Contribution to this document
Justin (oxSardius)	Architecture, requirements, product and go-to-market	Compiled the document and diagrams; led the design decisions and the red-team review
Sidharth	Anchor program (split with Yamin to be agreed at build start)	Team pass and self-check (input in the initial discussion); design decisions D1–D5; red-team decisions; approved the final plan and the follow-up review (2026-09-29)
Yamin Raad	Anchor program (split with Sidharth to be agreed at build start)	Team pass and self-check; design decisions D1–D5; red-team decisions; follow-up review of the red-team pass (Section 4.1)

All three members collaborate across areas. The program split between Sidharth and Yamin (for example, `join_duel` and escrow versus `swap` and the pool math) is set when the build starts.

Capstone idea (team decision). The team chose Rug Royale, from Justin’s LOI, as the group capstone.

Rug Royale is a 1v1 trading-duel protocol on Solana. Two players each escrow an entry, then trade an identical bankroll in the same StonkFun top-10 coin, in one shared pool, for a short fixed window. At the end, the program values both positions onchain and pays the pot minus rake to the higher PnL%. The MVP runs on devnet against a deterministic internal pool, with devnet-SOL entries and program-minted demo tokens.

1. Actors (carried forward from the LOI)

Category	Actor	Signs	Notes
Direct actor	Creator	<code>create_duel</code> , <code>swap</code>	Picks the duel coin from the top-10 snapshot, tier, window; escrows entry (D1)
Direct actor	Opponent	<code>join_duel</code> , <code>swap</code>	Joins on the creator’s coin; escrows entry (D1)
Direct actor	Sponsor	<code>sponsor_prize</code>	Funds the pot of an Open duel (freeroll)
Direct actor	Settler (any signer)	<code>settle</code> , <code>cancel_duel</code> , <code>close_duel</code>	Permissionless cranks; funds go to parties on record, never to the caller (except the optional tip)

Category	Actor	Signs	Notes
Direct actor	Crank bot (a declared Settler, run by the team)	<code>settle</code> , <code>cancel_duel</code> , <code>close_duel</code>	Off-chain script; sends <code>settle</code> within seconds of <code>end_ts</code> for faster payouts (REQ14). Safety does not depend on it: <code>settle</code> never expires, so anyone can settle later with the same result (P1)
Beneficiary	Winner	none	Receives the pot inside the settler's transaction
Beneficiary	Treasury	none	Receives rake inside <code>settle</code>
Beneficiary	Both players, Sponsor	none	Receive refunds and rent when another party signs
Beneficiary	Spectators	none	Read vault balances and the <code>Duel</code> account (client-side)
Administrator	Config admin	<code>init_config</code>	Single key in the capstone; multisig or governance PDA later
Stakeholder	StonkFun, coin creators, Raydium, Melee, Helius, SolEnrich, Turbin3	none	External interest; no transactions in the capstone

2. MVP use cases

2.1 Team design decisions (2026-09-28)

#	Decision	Effect on the design
D1	One coin, one pool, one duel. Both players trade the same coin, chosen from the StonkFun top 10. Rival coins move to the roadmap. If both players buy and hold, the first buyer wins; a true draw happens only when neither player trades (corrected by P2).	<code>coin_b</code> is removed; one <code>Pool</code> per duel. Reverses the LOI's rival-coin format.
D2	Pure internal pool. Prices move only through the two duelists' trades. No keeper, no oracle, no off-chain price signer. The StonkFun API, Dexscreener, and Birdeye feed only the client display and the coin picker.	REQ10 becomes an on-chain constraint. No new actor.

#	Decision	Effect on the design
D3	Top-10 snapshot. The admin writes a StonkFun top-10 snapshot into <code>Config.allowed_mints</code> at <code>init_config</code> and keeps it for the MVP.	<code>allowed_mints: [Pubkey; 10]</code> . <code>update_config</code> is roadmap.
D4	Real devnet-SOL entries, virtual coins. Friendly duels escrow devnet SOL; zero-entry duels use the same path. The coin and the bankroll are program-minted demo assets.	See 2.2.
D5	Three MVP use cases: <code>join_duel</code> , <code>swap</code> , <code>settle</code> . <code>create_duel</code> is a setup step, shown on the diagrams with REQ02.	Sections 2.3 to 2.5.

2.2 How devnet-SOL entries work without StonkFun coins on devnet (D4)

A duel holds three separate assets. Only the entry is the player's own money, and it never touches the coin.

Asset	What it is on devnet	Who creates it	What it does	Mainnet equivalent
Entry	Devnet SOL (lamports) from the faucet	The player	Sits in <code>Escrow</code> ; the winner receives the pot minus rake	SOL or USDC
Bankroll	Demo quote token standing in for STONK	<code>MintAuthority</code> PDA, at <code>join_duel</code>	Equal starting balance for both players; measures PnL%	The player's own STONK
Coin	Demo mint standing in for one top-10 StonkFun coin	<code>MintAuthority</code> PDA, pool seed at <code>create_duel</code>	The asset both players trade in the duel's pool	The real StonkFun coin on Raydium

The entry decides who gets paid. The bankroll and the coin decide who won. So `settle` compares PnL% in the demo tokens and pays out in real devnet SOL.

2.3 to 2.5 MVP use cases

Carried forward from LOI Phase 2 Part B (UC-4, UC-5, UC-6) and updated for D1 (one coin, one pool, one duel) and the follow-up review (Section 4.1).

Fields: Actor (signer) · Preconditions · Inputs · State transition · Accounts written · Transfers · Event · Typed errors · REQ IDs.

UC-1 `join_duel` (LOI UC-4) · REQ04

- **Actor:** Opponent (signer, payer).
- **Preconditions:** `status == Open`; `now < join_deadline`; `signer != creator`; `allowed_opponent` matches if set. The duel coin was validated against the top-10 snapshot at `create_duel`.
- **Inputs:** none. The opponent accepts the creator's coin, tier, window, and entry as recorded in `Duel`.
- **State transition:** `Open` → `Active`; sets `opponent`, `start_ts = now + 60`, `end_ts = start_ts + window_secs`.
- **Accounts written:** `Duel`; opponent vault authority and its quote and coin token accounts (init, Token-2022 interface); both quote vaults credited with the bankroll.
- **Transfers:** `entry_lamports` Opponent → `Escrow` (0 in unranked and freeroll); bankroll minted by `MintAuthority` into both quote vaults. No pool seed at join; the duel's one pool was seeded at `create_duel`.
- **Event:** `DuelJoined`.
- **Errors:** `DuelNotOpen`, `JoinDeadlinePassed`, `CannotJoinOwnDuel`, `OpponentNotAllowed`.

UC-2 `swap` (LOI UC-5) • REQ05, REQ10

- **Actor:** Duelist (signer; equals `creator` or `opponent`).
- **Preconditions:** `status == Active`; `start_ts <= now < end_ts`; the pool is the duel's one pool (`["pool", duel]`). Both players trade the same pool, so each player's trades move the price the other sees.
- **Inputs:** `side: Buy | Sell`, `amount_in`, `min_out`.
- **State transition:** `Pool`.{`quote_reserve`, `token_reserve`} updated by constant-product math with fee; vault balances updated. Status unchanged.
- **Accounts written:** `Pool`, player quote vault, player coin vault, pool quote and coin accounts.
- **Transfers:** PDA-signed `transfer_checked` (Token-2022 interface; transfer-fee mints supported) between vault and pool.
- **Event:** `SwapExecuted { duel, player, mint, side, in_amt, out_amt }`.
- **Errors:** `DuelNotActive`, `WindowNotStarted`, `WindowEnded`, `NotAParticipant`, `WrongPool`, `InsufficientBankroll`, `SlippageExceeded`, `MathOverflow`.

UC-3 `settle` (LOI UC-6) • REQ06

- **Actor:** Settler (any signer). In practice the crank bot within seconds of `end_ts`, or either player's client through the Settle button (RT-1). No PDA or timer sends the transaction. `settle` has no upper time limit, so a late settle gives the same result (P1).
- **Preconditions:** `status == Active`; `now >= end_ts`.
- **Inputs:** none.
- **State transition:** `Active` → `Settled`. For each player: `final = quote_vault_balance + simulated_sell(coin_vault_balance, duel pool reserves at end)`. Each player is valued independently against the same end-of-window reserves; no pool mutation, so the order of valuation does not change the result. `pn1 = final / bankroll`. Records `winner`, `final_a` (creator), `final_b` (opponent). Tie if equal. A true tie happens only when neither player trades; if both buy and hold, the first buyer wins (P2).
- **Accounts written:** `Duel`, `Escrow`, `Treasury`, winner wallet, settler wallet (tip), both players (tie), sponsor (tie).

- **Transfers:** $\text{pot} = 2 \times \text{entry} + \text{sponsored}$; $\text{rake} = \text{pot} \times \text{rake_bps}$ (0 when entry is 0); $\text{tip} = \min(\text{settler_tip_lamports}, \text{pot} - \text{rake})$; winner receives $\text{pot} - \text{rake} - \text{tip}$. Tie: entries back to each player in full; $\text{tip} = \min(\text{settler_tip_lamports}, \text{sponsored})$, taken from the sponsored amount only; the rest of the sponsored amount split equally; no rake. With no sponsor, no tip on a tie (P5).
 - **Event:** `DuelSettled { duel, winner, final_a, final_b, prize }` (notification only; the result is stored in the `Duel` account, RT-7). The client shows You Win! / You Lose! when `Duel.status` is `Settled`, read on page load and through `accountSubscribe` (RT-8, P5).
 - **Errors:** `DuelNotActive`, `WindowNotEnded`, `MathOverflow`.
-

3. Granularity self-check (team, before any AI)

3.1 Team initial assessment (handwritten, verbatim, 2026-09-28)

1. Proof of Concept

a. Main use case if the quintessential duel at the heart of rug royale: two stonkfun traders who want to flex their PnL enter a duel (either of same coin, or a coin in the stonkfun top10 (using the stonk API to select at exact time of `create_duel`, this either picks the same coin from the top 10 list, or picks a variety, we need to workshop this more). This initiates the escrow pot between duellists. The creator who initiates the duel signs `create_duel`, (see granularity below) then the opponent signs `join_duel`, and adds fund to escrow. We can do virtual free-roll duels to start to keep it simple, also I'm unsure if devnet covers stonkfun RWAs, (assumption what devnet covers to see if we would even be able to do a non-virtual contest). For the MVP, the spectator beneficiary stuff should be deprioritized, unless we see an emergent use case through developed our mainline MVP. It will be challenging to have live spectators of trading events and charts in an MVP, we should save that for PMF. Swap and settle will need to be initiated post duel, plus a win condition and popup through the UI of the app.

2. Granularity self-check

- a. Atomicity – each use case represents exactly one instruction handler (Yes, confirmed in LOI architecture)
- b. State Ownership – also confirmed Yes in LOI architecture
- c. Backend assumptions – I don't think there are any, but we need to confirm in our adversarial test phase.
- d. Onchain vs. offchain
 - i. Protocol logic is separated into all the actor categories specified in the LOI. These all list program logic for each granular instruction capability
 - ii. Offchain protocol – Front end Vercel Next.js app, alerts delivered client side based on onchain events once the duels end and are settled, triggered by a onchain event, stonkfun API will give us prices of coins for the duels, and up to date price changes, we can also utilize dexscreener (and birdeye as a fallback) for price action if needed. Assumption: Do we need Pyth for on vs. offchain integration? I think no, but let's confirm. Client will also give a cool You Win! Pop up, and a devastating You Lose! Pop up once the

duel is settled. We should have a good UX for this so the user feels something.

3. Direct actors (signers who invoke instructions)
 - a. Creator signs `create_duel` (picks coin A, tier, window, escrows entry).
 - i. One instruction located in part B of LOI
 - b. Opponent signs `join_duel` (picks coin B, escrows entry).
 - c. Duelist (Creator or Opponent) signs swap during the window, on their own coin only.
 - d. Sponsor signs `sponsor_prize` to fund the pot of an Open duel (freeroll).
 - e. Settler is any signer who cranks `settle` after `end_ts`; may receive the optional tip.
 - f. Any signer may call `cancel_duel` (Open past deadline), `end` plus grace), and `emergency_refund` (Active past `close_duel` (terminal)). Funds always route to the parties on record, never to the caller.
 - g. Admin signs `init_config` once. No other instruction requires the admin. Use case `maxim`
 - i. Stonkfun callout trader as the first persona
4. Use cases
 - a. These seem to all be present except for spectator stuff. Let's run a sanity check to confirm and correlate with LOI.
 - b. Top 1-3 (besides admin stuff like `init_config`) would be `create_duel`, `join_duel`, swap and settle. Looks like we have 4, I don't see how we can't like do the contest without swap, and send the user funds, those seem essential, unless I'm missing something?
5. Same coin vs. different coins?
 - a. Team wants same coin
 - i. Keep it stonkfun top 10 to pick
 - b. Different coins for future

3.2 Self-check per MVP use case

Team answers, verbatim (2026-09-28), before the AI pass.

Rule	UC-1 <code>join_duel</code>	UC-2 <code>swap</code>	UC-3 <code>settle</code>
1. Atomicity	" <code>join_duel</code> only does open to active, it is predicated on setup work being done, it doesn't have setup work that belongs within it."	"One call makes exactly one trade."	" <code>settle</code> only does active to settle, close account is a separate program."
2. State ownership	"Opponent, <code>start_ts</code> and <code>end_ts</code> are all stored as programed owned accounts, and the bankroll is in an offchain indexer."	"The pool reservers live in an offchain indexer as part of freeroll, player escrow balances exist as a devnet account."	"winner, <code>final_a</code> , and <code>final_b</code> are recorded in the event (we think)."

Rule	UC-1 join_duel	UC-2 swap	UC-3 settle
3. Real signers	“The creator signs the bankroll mint_to as part of the initial setup.”	“Swap is restricted to the opponent for that instance of the duel, no others can come in a modify.”	“Settle is time based, no caller settles it besides the PDA we beleive.”
4. On-chain vs client-side	“Countdown is clientside, unless this should be made onchain.”	“The price chart is offchain through an API for this MVP use case. Mainnet could be all onchain.”	“The pop-up is client side and offchain triggered on the event and seen through the UI”

4. Adversarial analysis (AI second pass)

Run only after Section 3 is complete.

AI pass run 2026-09-28 against the use cases in Section 2 and the self-check in Section 3.2. The findings are kept as the AI wrote them. “Team decision” records the team’s Accept or Reject, and any later amendment from the follow-up review (Section 4.1).

AI findings

#	UC	Rule	AI finding	Proposed change	Team decision
RT-1	UC-3 settle	3. Real signers	Nothing on Solana runs by itself at a time. A PDA cannot sign or send a transaction; it can only authorize a CPI inside a transaction someone else sent. “Settle is time based” hides a real question: who sends settle? If the answer is “our server”, that is a hidden backend.	State the signer: any wallet, in practice the winner’s client (a Settle button that appears at end_ts). The optional settler tip pays a third party to crank. emergency_refund covers the case where nobody settles. The payout goes to Duel.creator or Duel.opponent, checked by account constraints, so the caller cannot redirect it.	Accept. Team asked: what if the winner goes offline? Led to RT-14. Amended by P1: the line “emergency_refund covers the case where nobody settles” is wrong, because a permissionless settle never expires.

#	UC	Rule	AI finding	Proposed change	Team decision
RT-2	UC-1 join_duel	3. Real signers	No human signs the bankroll mint_to. The creator is not a signer in the opponent's join_duel transaction. The MintAuthority PDA authorizes the mint inside the program (invoke_signed). This is the property that removes a hidden minting key.	Rewrite the answer: the Opponent is the only signer; MintAuthority PDA signs the mint by seeds.	Accept.
RT-3	UC-1 join_duel	2. State ownership	The bankroll is on-chain, not in an indexer. It is a token balance in each player's quote vault (a token account whose authority is the vault PDA), and its size is Duel.bankroll.	Rewrite the answer: bankroll = token balance in the program-controlled quote vault; amount recorded in Duel.	Accept.
RT-4	UC-2 swap	2. State ownership	Pool reserves are on-chain, and they must be. They are the Pool PDA plus its two token accounts. Under D2 and REQ10, the reserves are the only price source for settle. If they lived in an indexer, settlement would depend on off-chain data and fail Rule 2 and Rule 3. Also, swap does not touch Escrow: it moves tokens between the player's vaults and the pool. Escrow holds only entry lamports.	Rewrite the answer: reserves in Pool + pool token accounts; player balances in the player's quote and coin vaults; escrow untouched by swap.	Accept.

#	UC	Rule	AI finding	Proposed change	Team decision
RT-5	UC-2 swap	3. Real signers	Both duelists can swap, not only the opponent. The check is <code>signer ∈ {Duel.creator, Duel.opponent}</code> , and the vault PDA is derived from the signer's key (<code>["vault", duel, signer]</code>), so a player can only move their own vault.	Rewrite the answer as the two-player check plus the seed binding.	Accept.
RT-6	UC-2 swap	4. On-chain vs client-side	The in-duel price chart cannot come from the StonkFun API. Under D2 the duel trades a demo coin in its own pool, so the real StonkFun price is a different number. The duel chart must plot the pool price, read from <code>Pool</code> reserves or <code>SwapExecuted</code> events via RPC. The API and Dexscreener can show the real coin as context only. “Mainnet could be all onchain” also needs care: on mainnet, settlement reads the chain; the chart stays client-side.	Rewrite the answer: duel chart = pool price via RPC (client); real-coin chart = API (client, context only); settlement = on-chain reserves.	Accept.
RT-7	UC-3 settle	2. State ownership	The result is in the Duel account, not the event. <code>settle</code> writes <code>winner</code> , <code>final_a</code> , <code>final_b</code> , and <code>status</code> into <code>Duel</code> . The event is a notification: logs can be truncated and are not a queryable record. LOI decision: the <code>Duel</code> account is the oracle.	Rewrite the answer: result stored in <code>Duel</code> (never closed); <code>DuelSettled</code> is a notification.	Accept.

#	UC	Rule	AI finding	Proposed change	Team decision
RT-8	UC-3 settle	4. On-chain vs client-side	The popup should trigger on the account, not the event. For the same reason as RT-7, the client subscribes to the Duel account and shows the popup when status becomes Settled.	Trigger You Win! / You Lose! from Duel.status via accountSubscribe; the event is optional.	Accept. Extended by P5: also read Duel.status on page load, so a player who was offline at settlement still sees the result.
RT-9	UC-3 settle	1. Atomicity	Wording: close_duel is a separate instruction handler in the same program, not a separate program. The answer is correct in substance.	Replace “program” with “instruction handler”.	Accept.
RT-10	UC-1 join_duel	4. On-chain vs client-side	The countdown answer is correct. The timer display is client-side; the enforcement is on-chain, because swap checks start_ts <= now < end_ts against the Clock sysvar. Nothing needs to move on-chain.	Add the second half: enforcement on-chain via start_ts and end_ts.	Accept.
RT-11	UC-1 join_duel	1. Atomicity	Possible attack: join_duel does four things (escrow the entry, create vaults, mint two bankrolls, set the window). Should the bankroll mint be a separate fund_bankroll handler?	AI view: keep it in one handler. Splitting creates an Active duel with unfunded vaults, a state that must then be guarded everywhere. One transition = one handler, not one line of work. The team can accept or reject.	Accept the AI view: the bankroll mint stays inside join_duel. (Relabelled from Reject by P4: the team agreed with the AI here.)
RT-12	UC-3 settle	Design (outside the 4 rules)	Design suggestion: a player with zero swaps forfeits, to reduce buy-and-hold draws (D1).	One check in settle. The team may reject for the MVP: holding is a legal strategy, and draws cost nothing in freeroll.	Reject. No forfeit rule. P2 later showed the premise was wrong: buy-and-hold is not a draw, because the first buyer wins. The rejection stands.

#	UC	Rule	AI finding	Proposed change	Team decision
RT-13	UC-2 <code>swap</code>	Design (outside the 4 rules)	Two-player pool dynamic: in one shared pool, a player can watch the other's trades and trade against them (for example, buy right before selling into the opponent's buy). This is not a bug, but it means the duel is a game of reading the opponent, not the wider market.	No program change. State it in the rules and on the duel screen.	Reject the warnings. No program or UI change.
RT-14	UC-3 <code>settle</code>	3. Real signers	Raised by the team's question on RT-1: "What happens if the winner's power goes out, and they won, but the UI glitches and they can't settle?" The late time does not change the result: <code>swap</code> stops at <code>end_ts</code> , so balances and reserves freeze and <code>settle</code> computes the same winner at any later time, paid to the recorded winner. But <code>settle</code> and <code>emergency_refund</code> are both permissionless, and after <code>end_ts + grace_secs</code> both are valid. If nobody settles in time, the loser can call <code>emergency_refund</code> , take back their entry, and cancel the winner's win.	Options: (A) long <code>grace_secs</code> (days) plus a crank bot that settles within seconds of <code>end_ts</code> ; (B) remove <code>emergency_refund</code> from the MVP; (C) make <code>emergency_refund</code> pay out like <code>settle</code> . AI recommends A. Pyth does not apply: it is a price oracle and cannot send or schedule transactions.	Accept A (2026-09-28). Superseded by P1 (2026-09-29): option A only made the race unlikely; option B removes it. <code>emergency_refund</code> is removed from the MVP.

What the AI flagged that we decided was wrong

#	AI finding	Why we rejected it
RT-12	A player with zero swaps forfeits, to reduce buy-and-hold draws	"Buying and holding is a legitimate strategy the team agreed you should be able to do."

#	AI finding	Why we rejected it
RT-13	Warn players that the opponent can trade against them in the shared pool	“That is a feature, not a bug.” Reading and trading against the opponent is the game.
RT-14 (option A)	Keep <code>emergency_refund</code> with a 7-day grace period plus a crank bot	Yamin’s review (P1): the program still allowed the loser-refund race; safety depended on the bot staying up. <code>settle</code> never expires, so the refund path added only risk.

4.1 Team follow-up review (Yamin Raad, 2026-09-29)

Yamin reviewed RT-1 to RT-14 after the AI pass. The review found two errors in the AI’s own findings and one design issue that neither the team nor the AI had caught.

#	Finding	Team decision	Change made
P1	Drop <code>emergency_refund</code> (RT-14, RT-1). After <code>end_ts</code> + <code>grace_secs</code> , both <code>settle</code> and <code>emergency_refund</code> are valid. The program cannot tell “settle has a bug” from “nobody has called settle yet”, so the loser can refund and cancel the win. The 7-day grace and the bot only make this unlikely. <code>settle</code> is deterministic and never expires, so without a refund path there is nothing to race. RT-1’s line “ <code>emergency_refund</code> covers the case where nobody settles” is wrong for the same reason.	Accept: drop it. Alternatives considered and not chosen for the MVP: a refund that both players sign, and a fallback inside <code>settle</code> that refunds the entries if its checked math fails.	9 → 8 handlers; <code>grace_secs</code> , the <code>Refunded</code> state, and <code>REQo8</code> removed; the crank bot becomes a convenience for faster payouts. A <code>settle</code> bug on devnet is fixed with a program upgrade; a mainnet escape hatch is planned later.
P2	Buy-and-hold is not a draw (D1, UC-3, RT-12). In one constant-product pool, the first buyer gets more coin for the same spend and wins (worked example in Section 7). Every duel opens with a race to land the first swap at <code>start_ts</code> , which favors bots and players paying priority fees. The only true draw is when neither player trades.	Accept: keep the opening race for the MVP and document it.	D1, UC-3, 6.3, 6.5, and Section 7 corrected. Opening batch auction added to the roadmap.

#	Finding	Team decision	Change made
P3	A creator can take a sponsor's prize (REQ03, new). The creator sets <code>allowed_opponent</code> to a second wallet they control, joins with it, and is guaranteed the sponsored pot.	Accept as a known MVP risk. <code>sponsor_prize</code> is not an MVP use case and the team is the only devnet sponsor.	Recorded in Section 7. Mainnet fix: the sponsor approves the opponent, plus SolEnrich collusion signals.
P4	Relabel RT-11 as Accept. The AI recommended keeping the bankroll mint inside <code>join_duel</code> , which is what the team decided.	Accept.	RT-11 moved out of “What the AI flagged that we decided was wrong”.
P5	Small fixes. (a) Read <code>Duel.status</code> on page load as well as through <code>accountSubscribe</code> . (b) At exactly <code>now == join_deadline</code> , neither <code>join_duel</code> nor <code>cancel_duel</code> succeeds; use <code>now >= join_deadline</code> for cancel. (c) On a tie the entries return in full, so the tip needs a source.	Accept all three. Tip on a tie comes from the sponsored amount only; with no sponsor, no tip.	UC-3, REQ06, REQ07, 6.3, 6.5, and the constraints updated.

No change needed (per the review): RT-2, RT-3, RT-4, RT-5, RT-6, RT-7, RT-9, RT-12, and RT-13 are valid as written.

4.2 Final granularity check (after all accepted findings)

Rule	UC-1 <code>join_duel</code>	UC-2 <code>swap</code>	UC-3 <code>settle</code>
1. Atomicity	Pass. One transition, <code>Open</code> → <code>Active</code> . Entry escrow, vault creation, and bankroll mint happen in the same transaction, so the duel can never be <code>Active</code> with empty vaults (RT-11).	Pass. One call, one trade, no status change.	Pass. One transition, <code>Active</code> → <code>Settled</code> . Cleanup is the separate <code>close_duel</code> handler (RT-9).
2. State ownership	Pass. <code>opponent</code> , <code>start_ts</code> , <code>end_ts</code> , <code>bankroll</code> in <code>Duel</code> ; bankroll tokens in program-controlled quote vaults (RT-3).	Pass. Reserves in the <code>Pool</code> PDA and its token accounts; balances in the player's vaults; <code>Escrow</code> untouched (RT-4).	Pass. <code>winner</code> , <code>final_a</code> , <code>final_b</code> , <code>status</code> in <code>Duel</code> , which is never closed; the event is a notification (RT-7).

Rule	UC-1 join_duel	UC-2 swap	UC-3 settle
3. Real signers	Pass. Opponent is the only signer; the <code>MintAuthority</code> PDA authorizes the mint by seeds (RT-2).	Pass. Signer must be <code>Duel.creator</code> or <code>Duel.opponent</code> ; the vault PDA is derived from the signer's key (RT-5).	Pass. Any signer: crank bot or either player's client. Payout fixed to the recorded players. <code>settle</code> never expires and no refund path competes with it, so a late settle cannot lose the win (RT-1, P1).
4. On-chain vs client-side	Pass. Countdown display is client-side; the window is enforced on-chain by <code>start_ts</code> and <code>end_ts</code> (RT-10).	Pass. Duel chart plots the pool price via RPC; StonkFun API and Dexscreener are context only; settlement reads on-chain reserves (RT-6).	Pass. Popup driven by <code>Duel.status</code> , read on page load and via <code>accountSubscribe</code> (RT-8, P5); crank bot is an off-chain script (REQ14).

5. Requirements (REQ IDs)

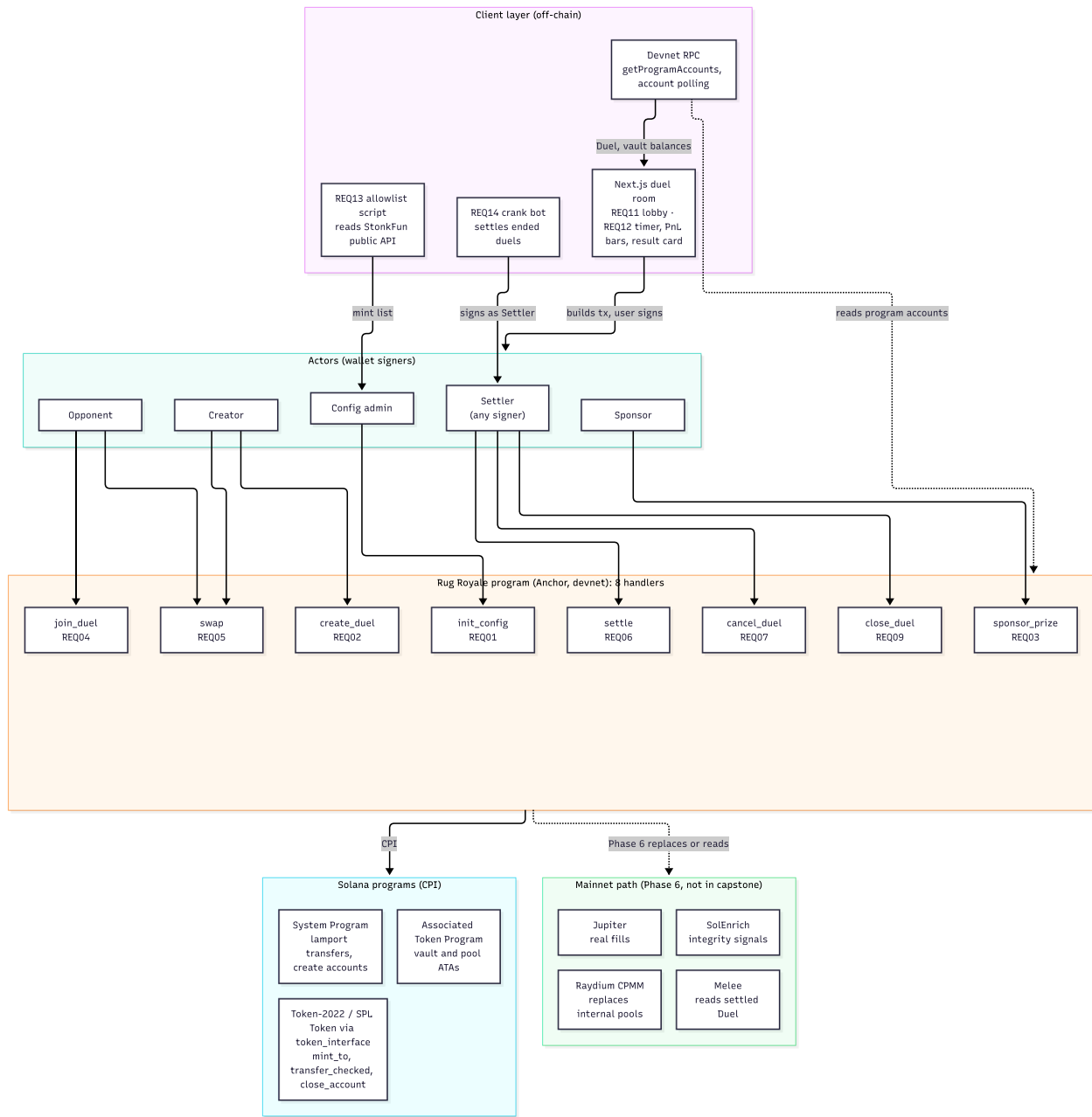
Each REQ ID appears on the diagrams next to the arrow or handler that carries it out. REQ08 is kept as a removed number so the other IDs stay stable.

REQ	Requirement	Handler or component	Layer
REQ01	Store protocol tunables (treasury, rake, tip, tiers, windows, pool seed ratio, quote mint, top-10 coin snapshot) in one program account	<code>init_config</code>	On-chain
REQ02	Open a duel: validate coin, tier, window, deadline; escrow the creator's entry; create creator vaults and the duel's one pool	<code>create_duel</code>	On-chain
REQ03	Fund a freeroll pot from one sponsor while the duel is Open (known MVP risk: creator self-dealing, P3)	<code>sponsor_prize</code>	On-chain
REQ04	Seat the opponent: escrow the entry; mint equal bankrolls into both quote vaults; set the window; Open → Active	<code>join_duel</code>	On-chain
REQ05	Trade inside the window, in the duel's one pool, with constant-product math, fee, and slippage limit	<code>swap</code>	On-chain

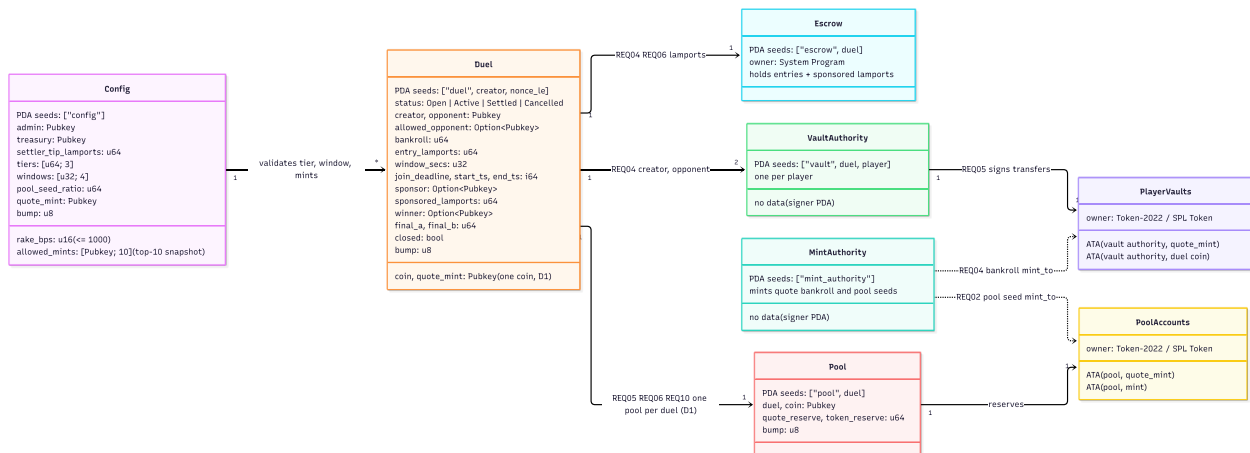
REQ	Requirement	Handler or component	Layer
REQ06	Value both positions by simulated sell without pool mutation; pay winner, rake, and tip; handle a tie (tip from sponsored only); no upper time limit; Active → Settled	<code>settle</code>	On-chain
REQ07	Refund an unjoined duel once <code>now >= join_deadline</code> ; Open → Cancelled	<code>cancel_duel</code>	On-chain
REQ08	<i>Removed from the MVP (P1).</i> Formerly <code>emergency_refund</code>	none	—
REQ09	Close vaults, pools, and escrow; return rent to payers; keep the <code>Duel</code> account as the permanent record	<code>close_duel</code>	On-chain
REQ10	Price comes only from the duel pool's reserves; no oracle, keeper, or off-chain price signer (D2)	<code>swap</code> , <code>settle</code>	On-chain
REQ11	List open duels (lobby)	<code>getProgramAccounts</code> on devnet; indexer on mainnet	Client
REQ12	Show live PnL bars, timers, the result card, and the result popup (read on page load and via subscription)	RPC reads of vaults and <code>Duel</code>	Client
REQ13	Take the StonkFun top-10 snapshot from the public API; show coin prices with Dexscreener or Birdeye (display only)	Off-chain script; admin writes <code>Config.allowed_mints</code> once (D3)	Client
REQ14	Settle ended duels within seconds of <code>end_ts</code> for faster payouts; safety does not depend on it (P1)	Crank bot (declared Settler) calling <code>settle</code>	Client

6. Architecture diagrams

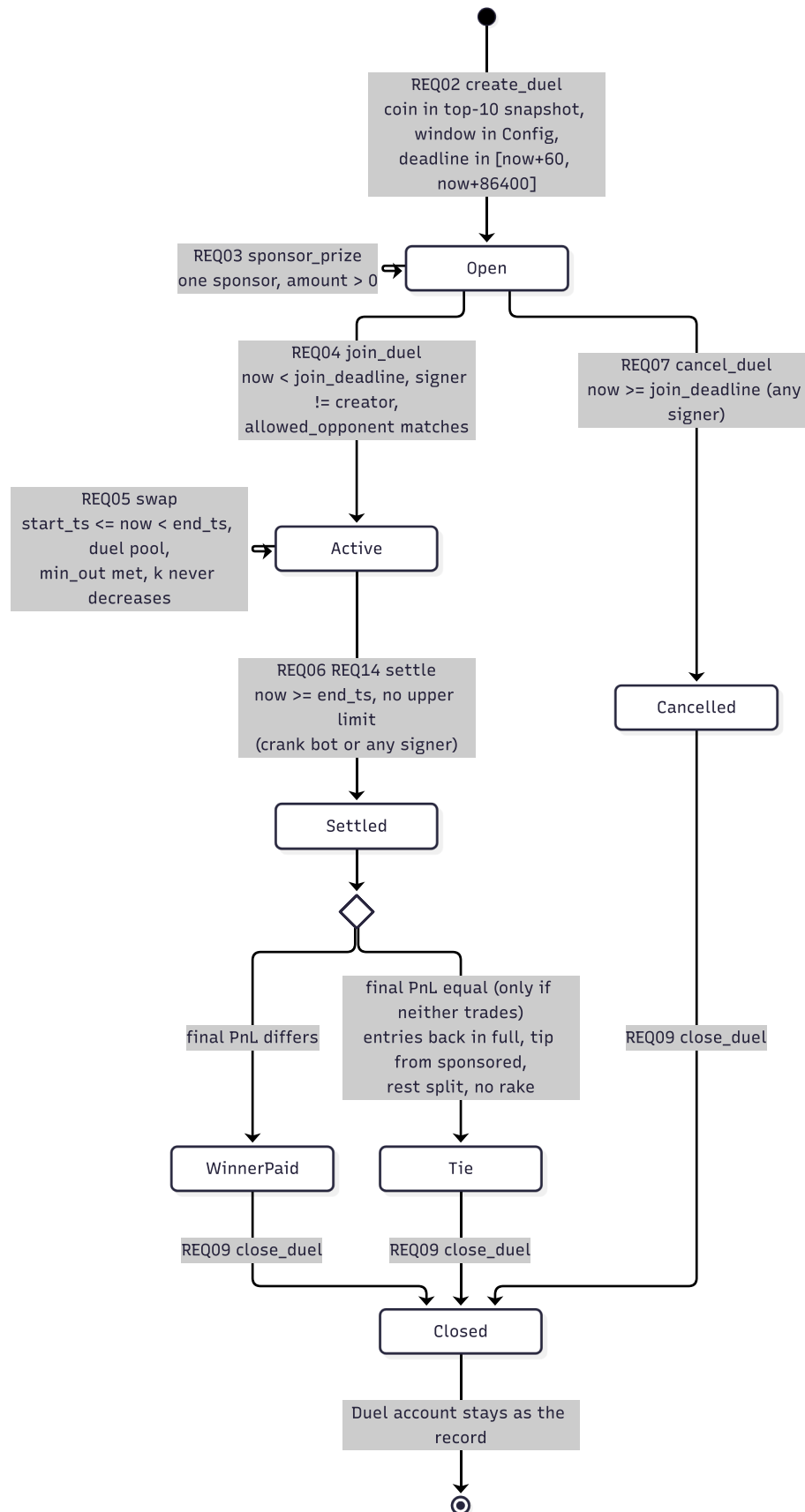
6.1 System context: programs, actors, client, external dependencies



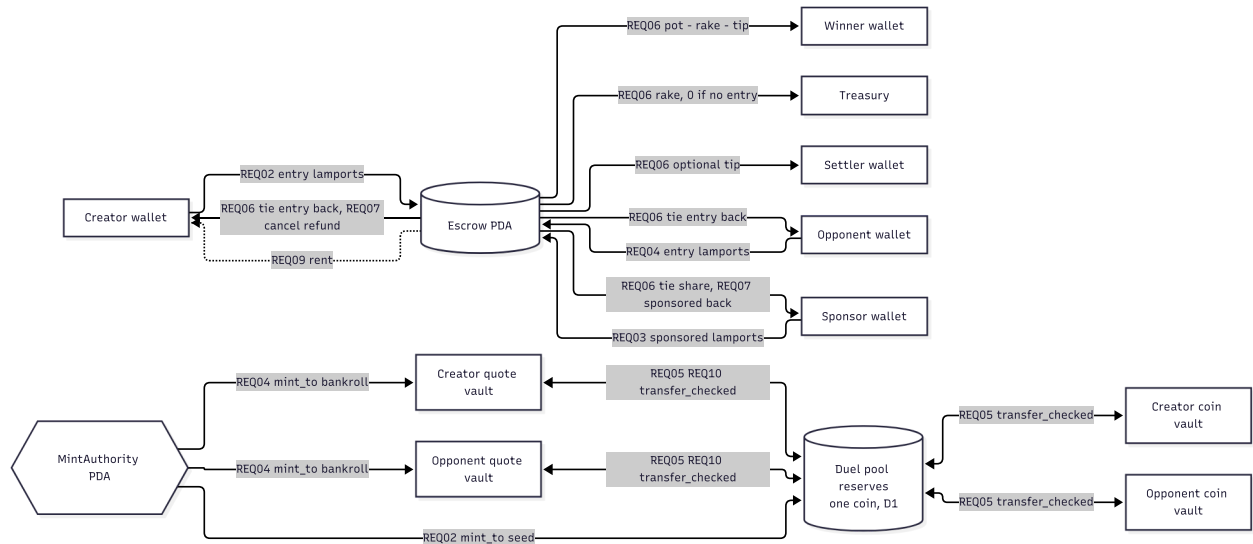
6.2 Account model: PDAs, seeds, and fields



6.3 Duel lifecycle: states, validation points, alternate flows

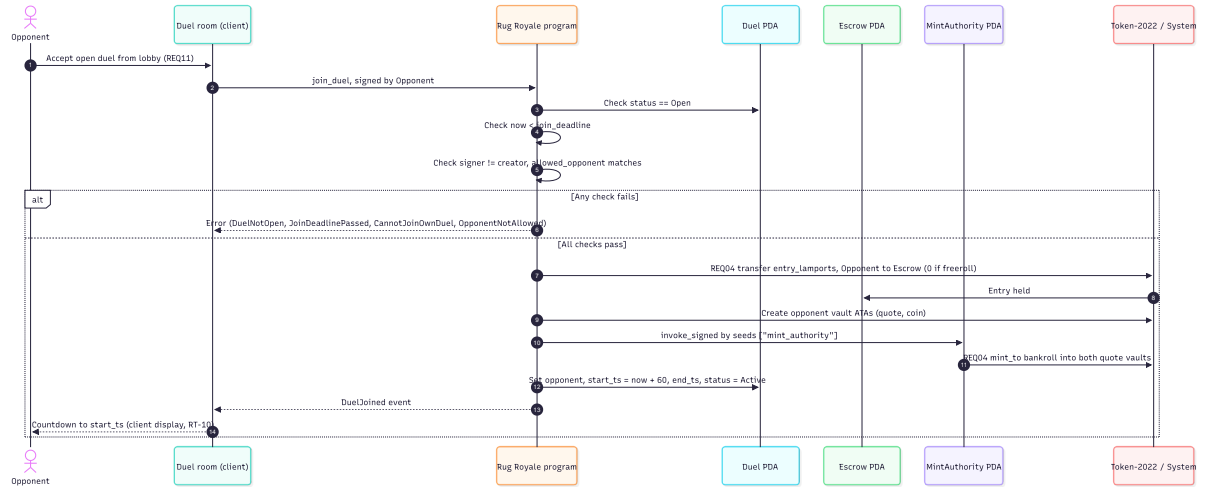


6.4 Fund and token flows

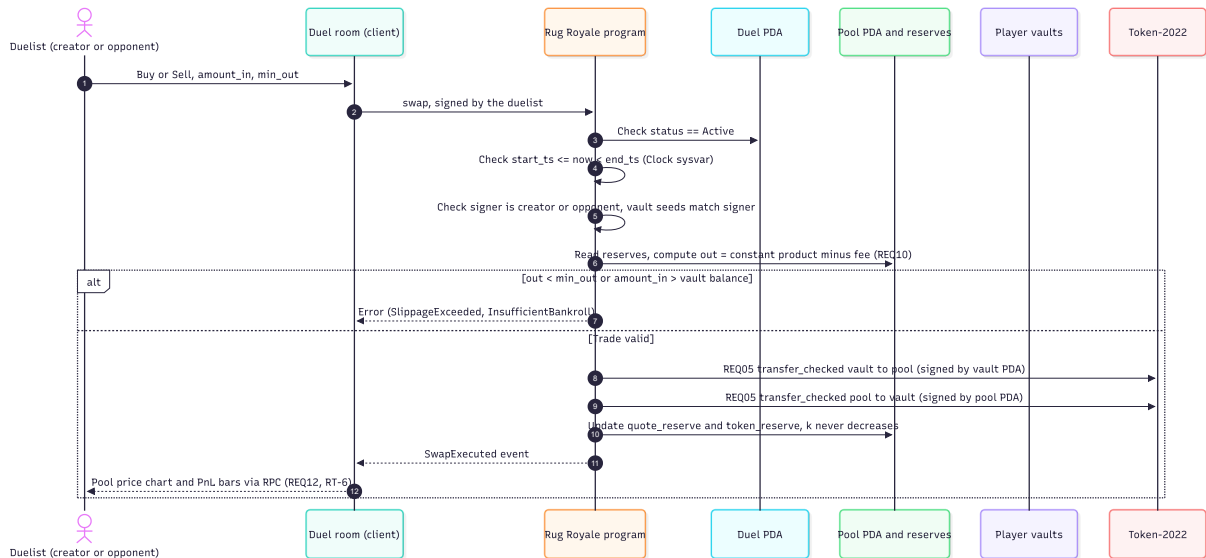


6.5 MVP sequences: one per use case, with validation checks

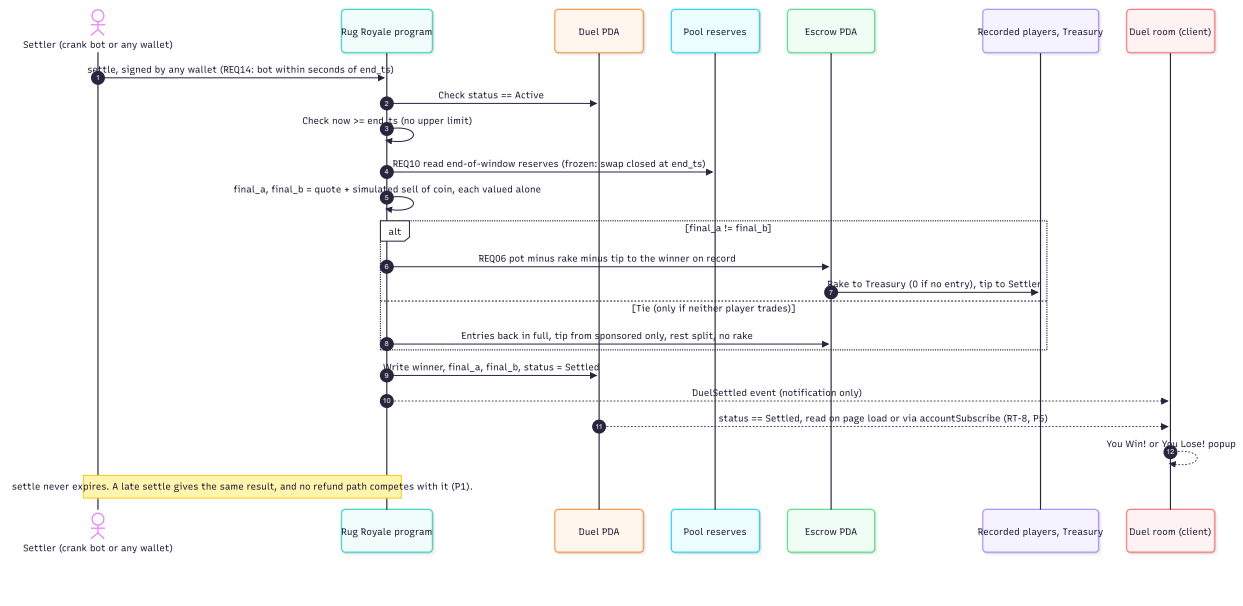
UC-1 join_duel (REQ04)



UC-2 swap (REQ05, REQ10)



UC-3 settle (REQ06, REQ14)



7. Design decisions and known MVP limits

7.1 Devnet price movement (decided, D2)

With one coin and one pool per duel, both duelists trade the same reserves. Each player's trades move the price the other player sees, so the duel is a two-player market. The price comes only from the pool reserves (REQ10). No oracle, keeper, or off-chain price signer exists, so no trusted party can move the price.

Consequences: - The market is closed. PnL for one player comes from the other player's trades, minus fees. - Real StonkFun price action (API, Dexscreener, Birdeye) appears in the client for context only. It never enters settlement.

Rejected for the MVP: a keeper actor that replays real prices (adds a trusted signer), shared per-coin pools across duels (needs many concurrent duels), and a Surfpool mainnet fork (not deterministic). The Surfpool fork stays as the stretch demo.

7.2 Known limit: the opening race (P2)

In one constant-product pool, the first buyer gets more coin for the same spend. Worked example (Yamin Raad): pool seeded at 100 quote / 100 coin, both players buy with 10 quote, fees ignored.

Player	Order	Coin received	Final value at settle	PnL
A	First	9.091	101.80	+1.80%
B	Second	7.576	100.00	0.00%

So if both players buy and hold, the first buyer wins. A true draw happens only when neither player trades. Every duel therefore opens with a race to land the first swap at `start_ts`, which

favors bots and players who pay priority fees. The team accepts the race for the MVP. Roadmap mitigation: an opening batch, where both players submit an opening buy that fills at the same price.

7.3 Known risk: sponsor-prize self-dealing (P3)

A creator can set `allowed_opponent` to a second wallet they control, join with it, and collect the sponsored pot. The MVP accepts this risk: `sponsor_prize` is not an MVP use case, and the team is the only devnet sponsor. Mainnet fix: the sponsor approves the opponent, plus SolEnrich collusion signals. No on-chain check can prove that two wallets belong to different people.

7.4 No refund path after start (P1)

`emergency_refund` is removed. `settle` is deterministic and has no upper time limit, so an Active duel always ends in `Settled`. A `settle` bug on devnet is fixed with a program upgrade. A mainnet escape hatch (for example, a refund that both players sign) is a post-MVP decision.

8. On-chain requirements matrix

Accounts and PDAs

Account	Seeds	Owner	Purpose	REQ
Config	["config"]	program	Tunables and the top-10 coin snapshot (<code>allowed_mints: [Pubkey; 10]</code>)	REQ01
MintAuthority	["mint_authority"]	program	Mint authority of the devnet quote mint and the demo coin mints	REQ02, REQ04
Duel	["duel", creator, nonce_le]	program	State machine, players, coin, window, entries, sponsor, results; permanent record	REQ02 to REQ07, REQ09
Escrow	["escrow", duel]	System Program (PDA-signed)	Entries and sponsored lamports	REQ02, REQ03, REQ04, REQ06, REQ07
Vault authority	["vault", duel, player]	program	Signs transfers for one player	REQ04, REQ05
Player quote vault, player coin vault	ATA(vault authority, mint)	Token program	Bankroll and position	REQ04, REQ05, REQ06
Pool (1 per duel)	["pool", duel]	program	Constant-product reserves for the duel coin; the only price source	REQ02, REQ05, REQ06, REQ10

Account	Seeds	Owner	Purpose	REQ
Pool quote and coin accounts	ATA(pool, mint)	Token program	Reserves	REQ02, REQ05

Instruction handlers (8): `init_config` (REQ01), `create_duel` (REQ02), `sponsor_prize` (REQ03), `join_duel` (REQ04), `swap` (REQ05), `settle` (REQ06), `cancel_duel` (REQ07), `close_duel` (REQ09).

CPI dependencies: System Program (lamport transfers, account creation); Token-2022 and SPL Token via `token_interface` (`mint_to`, `transfer_checked`, `close_account`); Associated Token Program. No external DEX CPI in the capstone.

Custom constraints

- `rake_bps <= 1000`; rake is 0 when `entry_lamports == 0`.
- `window_secs ∈ Config.windows`; tier maps to `Config.tiers[tier]`.
- `coin ∈ Config.allowed_mints` (top-10 snapshot, fixed for the MVP); `coin != quote_mint`.
- `join_deadline ∈ [now + 60, now + 86400]`.
- `join_duel` requires `now < join_deadline`; `cancel_duel` requires `now >= join_deadline`, so exactly one of them is valid at any time (P5).
- `swap` only in the duel's pool; `amount_in <= vault balance`; constant-product with fee so `k` never decreases; `min_out` enforced.
- Settlement valuation is a pure function of vault balances and pool reserves (simulated sell); no order dependence.
- `settle` has no upper time limit; no other handler can move an Active duel out of Active (P1).
- One sponsor per duel; sponsored funds return to the sponsor on cancel. On a tie, the tip comes from the sponsored amount only and the rest is split equally; with no sponsor, no tip (P5).
- Every transfer PDA-signed with explicit seeds; checked math; no `unwrap()`; Token-2022 `transfer_checked` with fee-aware amounts.
- First valid `join_duel` wins the seat (status gate).
- `settle`, `cancel_duel`, `close_duel` are permissionless; the `Duel` account is never closed.
- Demo coin mint authority is the `MintAuthority` PDA (not burned), because every duel seeds new pools.

Explicitly off-chain: lobby (REQ11), live PnL, result card, and popup (REQ12), allowlist curation (REQ13), crank bot (REQ14), points and unranked leaderboard, streamer host links, SolEnrich collusion signals.

9. Traceability

Pick a use case and find it on every diagram (use case → diagram), or pick a REQ label on a diagram and find its use case or handler (diagram → use case).

MVP use cases → diagrams

Use case	REQ IDs	6.1 System context	6.2 Account model	6.3 Lifecycle	6.4 Fund flows	6.5 Sequence
UC-1 join_duel	REQ04	Opponent → join_duel REQ04	Duel → Escrow : REQ04, Duel → VaultAuthority : REQ04, MintAuthority ..> PlayerVaults : REQ04	Open → Active : REQ04 with its checks	REQ04 entry lamports to Escrow; REQ04 mint_to bankroll ×2	UC-1 sequence
UC-2 swap	REQ05, REQ10	Creator and Opponent → swap REQ05	Duel → Pool : REQ05 REQ10, VaultAuthority → PlayerVaults : REQ05	Active → Active : REQ05 with its checks	REQ05 REQ10 transfer_checked vault ↔ pool ×4	UC-2 sequence
UC-3 settle	REQ06, REQ14	Settler and REQ14 crank bot → settle REQ06	Duel → Escrow : REQ06, Duel → Pool : REQ06 (read only)	Active → Settled : REQ06 REQ14; winner/tie choice	REQ06 payout, rake, tip, tie refunds from Escrow	UC-3 sequence

All REQ IDs → where they appear

REQ	Handler or component	Use case	Diagrams
REQ01	init_config	Setup (outside MVP)	6.1
REQ02	create_duel	Setup (outside MVP)	6.1, 6.2, 6.3, 6.4
REQ03	sponsor_prize	Alternate flow (freeroll); known risk P3	6.1, 6.3, 6.4
REQ04	join_duel	UC-1	6.1, 6.2, 6.3, 6.4, 6.5
REQ05	swap	UC-2	6.1, 6.2, 6.3, 6.4, 6.5
REQ06	settle	UC-3	6.1, 6.2, 6.3, 6.4, 6.5
REQ07	cancel_duel	Alternate flow (no opponent)	6.1, 6.3, 6.4
REQ08	Removed (P1)	none	none
REQ09	close_duel	Cleanup	6.1, 6.3, 6.4
REQ10	Pool reserves as the only price source	UC-2, UC-3	6.2, 6.4, 6.5 (UC-2, UC-3)
REQ11	Lobby	Client, feeds UC-1	6.1, 6.5 (UC-1)
REQ12	Timer, PnL bars, result card, popup	Client, UC-2 and UC-3	6.1, 6.5 (UC-2)
REQ13	Top-10 snapshot script	Client, feeds REQ01	6.1
REQ14	Crank bot	Client, sends UC-3	6.1, 6.3, 6.5 (UC-3)